Ad Soyad: _____	**Tarih:** _____
Bölüm 12 – Ünite 4 – *Protokol Tasarımı – Çalışma Kağıdı*	

Süre

Bu çalışma kağıdını tamamlamak için tahmini süre: **35–45 dakika**.

Soru 1: Kavram – TCP `stream` (7 dakika – 18 puan)

İstemci aşağıdaki çağrıları sırasıyla yapsın:

```
send(s, "MERHABA",   7,   0);
send(s, "DUNYA",     5,   0);
send(s, "OBYS",      4,   0);
```

Sunucu tarafında ise:

```
char buf[1024];
int n = recv(s, buf, 1024, 0);   // tek cagri
```

a) Bu `recv` çağrısı tam olarak ne döndürebilir? En az 3 farklı senaryo yazın:

(a) _____

(b) _____

(c) _____

b) TCP'nin neden "stream" (akış) protokolü olarak adlandırıldığını tek cümleyle açıklayın:

_____

_____

c) Mesaj sınırlarını korumak *uygulamanın* sorumluluğundadır. Bu probleme *ne ad* verilir? _____

Soru 2: Senaryo – `\n` ile Çerçeveleme (7 dakika – 18 puan)

Bir OBYS protokolü mesajları `\n` (newline) ayracıyla çerçeveliyor. İstemci şu mesajı gönderiyor:

```
emit:LOG\nKritik Hata: dosya bulunamadi\nDevam ediyor\n
```

Yani *tek bir mantıksal mesajın* içinde `\n` karakterleri var.

a) Sunucu bu mesajı kaç parça olarak yorumlar? _____. Açıklayın:

_____

_____

b) Bu probleme genel adı nedir? _____

c) En az **iki farklı çözüm** önerin. (Sadece adlarını ve kısa açıklamasını yazın.)

(a) _____

_____

(b) _____

_____

Soru 3: Kod – Length-Prefix Decoder

(10 dakika – 25 puan)

Aşağıdaki şablonu **pseudo-kod** ile tamamlayın. Decoder bir akümülatör buffer (`tampon`) kullanır. Her gelen byte parçası tampon'a eklenir. Öndeki 4 bayt "mesaj uzunluğu" (uint32, ağ bayt sırası) sonrası gövde gelir. Tam mesaj toplandığında `teslim(mesaj)` çağrısı yapılır; tampon'da kalan baytlar bir sonraki çağrıya saklanır.

```
class LengthDecoder {
    QByteArray tampon;
    static constexpr int BASLIK_BOY = 4;
public:
    void verileriIsle(const QByteArray& yeni_parca) {
        tampon.append(yeni_parca);

        while (true) {
            // 1. Yeterli baslik var mi?
            if (_____________________________________________)  return;

            // 2. Uzunlugu oku (host byte order'a cevir)
            uint32_t boy_net = ___________________________________;
            uint32_t boy     = ntohl(boy_net);

            // 3. Tum govde gelmis mi?
            if (_____________________________________________)  return;

            // 4. Mesaji ayikla
            QByteArray mesaj = tampon.mid(_____, _____);

            // 5. Tampondan mesaji + basligi sil (kalan veriler korunsun)
            tampon.remove(_____, _____);

            // 6. Yukari kata teslim et
            teslim(mesaj);
        }
    }
};
```

İpucu

Tampon'da mesaj başlığının ilk byte'ı `tampon[0]`'dadır. Mesaj gövdesi `tampon[BASLIK_BOY]`'dan başlar. Tek bir TCP paketi içinde birden fazla mesaj olabilir; bu yüzden `while` döngüsü.

Soru 4: JSON Protokolü – OBYS

(7 dakika – 18 puan)

OBYS proje protokolünde aşağıdaki örnek istek var:

```
{
    "komut": "ogrenci_ekle",
    "veri":  { "no": "12345", "ad": "Ali Yilmaz" }
}
```

Bu basit format **üretim ortamında yetersiz**. Aşağıda eklenmesi önerilen alanlardan ikisini yazın ve görevini açıklayın.

1. **Alan adı:** _____
   Görevi: _____

_____

2. **Alan adı:** _____
   Görevi: _____

_____

İpucu

Düşünün: Sunucudan gelen cevabın hangi isteğe ait olduğunu nasıl bilirim? Protokolüm v1 mi v2 mi? İstemci eski sürüm mü?

Soru 5: Tasarım – Korelasyon ID'si

(7 dakika – 21 puan)

OBYS istemcisi sunucuya 5 farklı istek (öğrenci listesi, ders listesi, GPA, transkript, devamsızlık) gönderiyor. Sunucu cevapları **farklı sırada** dönebiliyor (asenkron işleme).

a) **Korelasyon ID'si** (correlation id, request id) nedir, kısaca tanımlayın:

_____

_____

b) Korelasyon ID'si olmasaydı ne olurdu? Somut bir örnekle açıklayın:

_____

_____

_____

c) İstemci tarafında 5 isteğin cevaplarını eşleştirmek için hangi veri yapısı uygundur? Anahtarı/değeri ne olur?

_____

d) Korelasyon ID'si üretmek için iki yaygın yöntem yazın:

(a) _____

(b) _____